Department: CSE AI & DS

Machine Learning Techniques

Lab Manual (2025-2026)

Prepared by: Prof. Rita Kadam

Practical No.1

Aim: To study and implement data preprocessing and feature engineering techniques.

Software Requirements: Python, Jupyter, NumPy, Pandas, Scikit-learn.

Objectives: Handle missing data, encoding, scaling, feature selection.

Theory:
In machine learning, the quality of data directly affects the performance of models. Real-world datasets are often incomplete, noisy, inconsistent, and unstructured, making them unsuitable for direct use in learning algorithms.
Data preprocessing is the process of cleaning and transforming raw data into a usable format, while feature engineering involves creating, modifying, or selecting features to improve model performance.
Importance of Data Preprocessing

- Improves data quality
- Reduces noise and inconsistency
- Enhances model accuracy and generalization
- Ensures fair contribution of features
- Speeds up model training and convergence

Importance of Feature Engineering

- Extracts meaningful patterns from raw data
- Reduces dimensionality
- Improves interpretability
- Helps algorithms learn efficiently

1. Handling Missing Values
Missing values may arise due to data collection errors or incomplete records.
Techniques:

- Mean Imputation (numerical data)
- Median Imputation (robust to outliers)
- Mode Imputation (categorical data)
- Deletion (row/column removal)
- Predictive Imputation (using ML models)

Algorithms / Methods:

- SimpleImputer (Scikit-learn)
- Mean/Median/Mode substitution

Output:

- Dataset with no missing (NaN) values
- Improved data completeness

2. Handling Categorical Data (Encoding)
Machine learning models require numerical input.
Techniques:

- Label Encoding
- One-Hot Encoding
- Ordinal Encoding

Algorithms / Methods:

- LabelEncoder
- OneHotEncoder
- get_dummies()

Output:

- Categorical attributes converted into numerical format
- Increased number of columns in case of one-hot encoding

3. Feature Scaling

Scaling ensures all features contribute equally.

Techniques:

- Normalization (Min–Max Scaling)
- Standardization (Z-score scaling)
- Robust Scaling

Algorithms / Methods:

- MinMaxScaler
- StandardScaler
- RobustScaler

Output:

- Features scaled to common range
- Faster convergence of algorithms like k-NN, SVM, Gradient Descent

4. Handling Outliers

Outliers can distort model learning.

Techniques:

- IQR (Interquartile Range)
- Z-score method
- Winsorization
- Clipping

Algorithms / Methods:

- Statistical thresholding
- Boxplot analysis

Output:

- Reduced impact of extreme values
- More stable model predictions

Program:

```
# Import required libraries
import pandas as pd
import numpy as np
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import LabelEncoder, OneHotEncoder, StandardScaler

# Step 1: Load sample dataset (Titanic-like sample)
data = {
    'Age': [22, np.nan, 24, 35, np.nan],
    'Sex': ['male', 'female', 'female', 'male', 'female'],
    'Fare': [7.25, 71.83, 8.05, 53.10, 8.45],
    'Pclass': [3, 1, 3, 1, 3],
    'Survived': [0, 1, 1, 1, 0]
}

df = pd.DataFrame(data)
print("Original Dataset:\n", df)

# Step 2: Handle missing values (Mean Imputation)
imputer = SimpleImputer(strategy='mean')
df['Age'] = imputer.fit_transform(df[['Age']])

# Step 3: Encode categorical variable (Sex)
label_encoder = LabelEncoder()
df['Sex'] = label_encoder.fit_transform(df['Sex'])
```

```
# Step 4: Feature Scaling (Standardization)
scaler = StandardScaler()
df[['Age', 'Fare']] = scaler.fit_transform(df[['Age', 'Fare']])

# Step 5: Feature Engineering (Create new feature)
df['Family_Size'] = df['Pclass'] + 1

print("\nPreprocessed Dataset:\n", df)
```

Conclusion:

Data preprocessing and feature engineering are critical steps in the machine learning pipeline. Proper handling of missing values, encoding, scaling, and feature selection significantly improves model performance, accuracy, and generalization ability. A well-preprocessed dataset leads to reliable and efficient machine learning models

Assignment / Viva Questions:

1. What is data preprocessing?
2. Why is feature scaling required in machine learning?
3. What is the difference between normalization and standardization?
4. What is feature engineering?
5. Name any two techniques used to handle missing data.

Practical No.2

Aim: To perform Exploratory Data Analysis (EDA) and study statistical relationships using correlation and covariance on a sample dataset.

Software Requirements:

Python 3.x

Jupyter Notebook / Google Colab

Libraries: Pandas, NumPy, Matplotlib, Seaborn

Objectives:

1. To understand the structure and characteristics of a dataset
2. To analyze data using descriptive statistics
3. To visualize data distributions and patterns
4. To study relationships between variables using correlation and covariance

Theory:

Exploratory Data Analysis (EDA)

EDA is the process of analyzing datasets to summarize their main characteristics using statistical measures and visualizations. It helps in identifying patterns, trends, outliers, and relationships among variables before applying machine learning models.

Descriptive Statistics

Descriptive statistics provide numerical summaries of data such as:

- Mean
- Median
- Standard Deviation
- Minimum and Maximum values

Correlation

Correlation measures the strength and direction of a linear relationship between two variables.

- Range: –1 to +1
- +1 → Perfect positive correlation
- –1 → Perfect negative correlation
- 0 → No linear relationship

Types of Correlation:

- Positive

- Negative
- Zero

Covariance

Covariance indicates the direction of relationship between two variables.

- Positive covariance → variables increase together
- Negative covariance → one increases, the other decreases
- Magnitude is not standardized (unlike correlation)

Algorithm:

1. Import required Python libraries.
2. Load the dataset into a Pandas DataFrame.
3. Display dataset structure and summary statistics.
4. Visualize data using histograms and boxplots.
5. Compute correlation matrix.
6. Compute covariance matrix.
7. Interpret the statistical relationships.

Sample Dataset

Iris Dataset

- Features: Sepal Length, Sepal Width, Petal Length, Petal Width
- Type: Multiclass dataset

Program:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load Iris dataset
df = sns.load_dataset('iris')

# Display first few records
print("Dataset Preview:\n", df.head())

# Descriptive statistics
print("\nDescriptive Statistics:\n", df.describe())
```

```
# Correlation matrix
corr = df.corr()
print("\nCorrelation Matrix:\n", corr)

# Covariance matrix
cov = df.cov()
print("\nCovariance Matrix:\n", cov)

# Visualization
sns.pairplot(df, hue='species')
plt.show()
```

Output:

Output 1: Dataset Preview

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa

Conclusion: EDA provides deep insights into data structure and relationships. Correlation and covariance help in identifying dependent features, which is crucial for feature selection and model building

Assignment/Viva Questions:

1. What is EDA?
2. What is correlation?
3. What is covariance?
4. What is the range of correlation coefficient?
5. Name one visualization technique used in EDA.

Practical No.3

Aim: To design and implement a Logistic Regression model for solving a classification problem using a sample dataset**.**

Software Requirements: • Python 3.x

- Jupyter Notebook / Google Colab
- Libraries:
  - NumPy
  - Pandas
  - Scikit-learn
  - Matplotlib

Objectives:

1. To understand the concept of binary classification
2. To implement Logistic Regression using Python
3. To train and test the classification model
4. To evaluate model performance using standard metrics

Theory: Logistic Regression

Logistic Regression is a **supervised machine learning algorithm** used for **classification problems**, especially binary classification. Unlike linear regression, logistic regression predicts the **probability** of an instance belonging to a particular class.

Sigmoid Function

The output of logistic regression is passed through a **sigmoid function**:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- Output range: **0 to 1**
- Probability ≥ 0.5 → Class 1
- Probability < 0.5 → Class 0

Decision Boundary

The model learns a decision boundary that separates different classes based on input features.

Advantages

- Simple and interpretable
- Efficient for linearly separable data
- Produces probabilistic output

Algorithm:

1. Import required Python libraries.
2. Load the sample dataset.
3. Separate input features and target labels.
4. Split the dataset into training and testing sets.
5. Initialize the Logistic Regression model.
6. Train the model using training data.
7. Predict class labels for test data.
8. Evaluate the model using accuracy and confusion matrix.

Sample Dataset

Breast Cancer Wisconsin Dataset

- Type: Binary classification
- Target: Malignant / Benign
- Features: Cell size, texture, radius, smoothness, etc.

Program :

```
# Import libraries

import pandas as pd

from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report


# Load dataset

data = load_breast_cancer()

X = data.data

y = data.target
```

```
# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

# Initialize Logistic Regression model
model = LogisticRegression(max_iter=5000)

# Train the model
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Evaluate model
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Output:
Output 1: Accuracy
Accuracy: 0.97
Interpretation:
The model correctly classifies 97% of the test samples.

Output 2: Confusion Matrix
[[52 2]
 [2 86]]

Actual / Predicted	Class 0	Class 1
Class 0	52	2
Class 1	2	86

Output 3: Classification Report

	precision	recall	f1-score	support
0	0.96	0.96	0.96	54
1	0.98	0.98	0.98	88
accuracy			0.97	142

Conclusion: Logistic Regression is an effective and computationally efficient algorithm for binary classification problems. It provides interpretable results and performs well when the classes are linearly separable

Assignment/Viva Questions:

1. What type of problem does Logistic Regression solve?
2. What is the range of the sigmoid function?
3. What does accuracy represent?
4. Name one evaluation metric for classification.
5. Is logistic regression a linear or non-linear model?

Practical No.4

Aim: To implement the **k-Nearest Neighbors (k-NN)** classification algorithm and analyze its performance using a sample dataset

Software Requirements: • Python 3.x

- Jupyter Notebook / Google Colab
- Libraries:
  - NumPy
  - Pandas
  - Scikit-learn
  - Matplotlib

Objectives:

- To understand the working principle of k-NN algorithm
- To implement k-NN for classification
- To analyze the effect of different values of **k** on performance
- To evaluate the model using standard performance metrics

Theory:

k-Nearest Neighbors (k-NN)

k-NN is a **supervised, instance-based learning algorithm** used for classification and regression. It does not build an explicit model; instead, it stores the training data and makes predictions based on similarity.

Working Principle

- Choose a value of **k** (number of neighbors)
- Calculate distance between the test sample and all training samples
- Select the **k closest neighbors**
- Assign the class based on **majority voting**

Distance Metrics

- Euclidean distance (most common)
- Manhattan distance
- Minkowski distance

Effect of k

- Small k → sensitive to noise (overfitting)
- Large k → smoother decision boundary (underfitting)

Algorithm

1. Import required Python libraries.
2. Load the sample dataset.
3. Separate features and class labels.
4. Normalize/scale the feature values.
5. Split the dataset into training and testing sets.
6. Select the value of **k**.
7. Train the k-NN classifier.
8. Predict class labels for test data.
9. Evaluate model performance using accuracy and confusion matrix.
10. Repeat for different values of **k** to analyze performance.

Sample Dataset:

Iris Dataset

- Features: Sepal length, Sepal width, Petal length, Petal width
- Classes: Setosa, Versicolor, Virginica

Program:

```
# Import libraries

import numpy as np

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.neighbors import KNeighborsClassifier

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import accuracy_score, confusion_matrix


# Load Iris dataset

data = load_iris()

X = data.data

y = data.target
```

```
# Feature scaling
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

# Train k-NN model with k = 5
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

# Make predictions
y_pred = knn.predict(X_test)

# Evaluate model
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

Output:

Output 1: Accuracy

Accuracy: 0.97

Interpretation:
The k-NN classifier correctly classifies 97% of the test samples.

Output 2: Confusion Matrix

```
[[12  0  0]
 [ 0 13  1]
 [ 0  0 12]]
```

Actual \ Predicted	Setosa	Versicolor	Virginica
Setosa	12	0	0
Versicolor	0	13	1
Virginica	0	0	12

Performance Analysis (Effect of k)

```
for k in range(1, 11):
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    print("k =", k, "Accuracy =", accuracy_score(y_test, pred))
```

Sample Output

k = 1 Accuracy = 0.95

k = 3 Accuracy = 0.97

k = 5 Accuracy = 0.97

k = 7 Accuracy = 0.95

k = 9 Accuracy = 0.94

Conclusion:

k-NN is a simple yet powerful classification algorithm that works well for small and well-scaled datasets. However, its performance strongly depends on the choice of **k**, distance metric, and feature scaling.

Assignment/Viva Questions:

1. Is k-NN a parametric or non-parametric algorithm?
2. What does the value of k represent?
3. Why is k-NN called a lazy learner?
4. Which distance metric is commonly used in k-NN?
5. What happens when k is very large?

Practical No. 5

Development of Predictive Models Using Multiple Linear Regression and Polynomial Regression on a Sample Dataset

---

Aim

To develop predictive models using Multiple Linear Regression and Polynomial Regression on a sample dataset and compare their performance for accurate prediction.

Objective

- To understand the concept of supervised learning for regression problems.
- To implement Multiple Linear Regression on a sample dataset.
- To implement Polynomial Regression on the same dataset.
- To compare the performance of both models using evaluation metrics such as Mean Squared Error (MSE) and R-squared (R^2) Score.
- To analyze which model is more suitable based on the nature of the dataset.

Theory

A. Introduction to Regression

Regression is a supervised machine learning technique used to predict continuous numerical values. It helps in identifying the relationship between independent variables (features) and a dependent variable (target).

Examples of regression problems:

- House price prediction

- Sales forecasting
- Temperature prediction
- Demand estimation

B. Multiple Linear Regression (MLR)

Multiple Linear Regression is an extension of simple linear regression where the dependent variable is predicted using two or more independent variables.

General Equation:

$$Y = b_0 + b_1X_1 + b_2X_2 + \cdots + b_nX_n$$

Where:

- Y= Dependent variable
- $X_1, X_2, \ldots, X_n$= Independent variables
- b_0= Intercept
- $b_1, b_2, \ldots, b_n$= Regression coefficients

Characteristics of MLR:

- Assumes a linear relationship between features and target.
- Easy to interpret.
- Works well when data follows a linear trend.
- Computationally efficient.

C. Polynomial Regression

Polynomial Regression is a special case of linear regression where the relationship between the independent variable and dependent variable is modeled as an nth-degree polynomial.

General Equation:

$$Y = b_0 + b_1X + b_2X^2 + b_3X^3 + \cdots + b_nX^n$$

Characteristics of Polynomial Regression:

- Captures nonlinear relationships in the data.
- Useful when a straight line cannot fit the data properly.
- Can model curves and bends in data.

- Higher polynomial degrees may cause overfitting.

Model Evaluation Metrics

(a) Mean Squared Error (MSE)

Measures the average squared difference between actual and predicted values.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- Lower MSE indicates better model performance.

(b) R-squared Score (R^2)

Measures how well the model explains the variance in the dependent variable.

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

- Value ranges from 0 to 1.
- Higher R^2 indicates better fit.

Requirements

- Python 3.x
- Jupyter Notebook / Google Colab
- Libraries:
  - NumPy
  - Pandas
  - Matplotlib
  - Scikit-learn
- Sample Dataset (.csv file)

Algorithm

1. Import the required Python libraries.
2. Load the sample dataset using Pandas.
3. Select input features (independent variables) and target variable (dependent variable).
4. Split the dataset into training and testing sets.
5. Train the Multiple Linear Regression model on the training data.
6. Predict target values using the testing data.
7. Transform the input features using PolynomialFeatures for a selected degree (e.g., degree = 2).
8. Train the Polynomial Regression model using transformed features.
9. Predict the target values for the polynomial model.
10. Calculate MSE and R^2 score for both models.
11. Compare the results and determine which model performs better.

Flowchart (Text Form)

Start
↓
Import Required Libraries
↓
Load Sample Dataset
↓
Select Features (X) and Target (Y)
↓
Split Dataset into Training and Testing Sets
↓
Train Multiple Linear Regression Model
↓
Predict Output and Evaluate
↓

Apply Polynomial Feature Transformation
↓
Train Polynomial Regression Model
↓
Predict Output and Evaluate
↓
Compare MLR and Polynomial Regression Results
↓
End

Python Program

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error, r2_score

# Load dataset
df = pd.read_csv("sample_dataset.csv")

# Example: assuming last column is target variable
X = df.iloc[:, :-1]
y = df.iloc[:, -1]

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ----------------------------
# Multiple Linear Regression
# ----------------------------
mlr_model = LinearRegression()
mlr_model.fit(X_train, y_train)



y_pred_mlr = mlr_model.predict(X_test)

mse_mlr = mean_squared_error(y_test, y_pred_mlr)
r2_mlr = r2_score(y_test, y_pred_mlr)

print("Multiple Linear Regression Results")
```

```
print("MSE:", mse_mlr)
print("R2 Score:", r2_mlr)

# ----------------------------
# Polynomial Regression
# ----------------------------
poly = PolynomialFeatures(degree=2)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)

poly_model = LinearRegression()
poly_model.fit(X_train_poly, y_train)

y_pred_poly = poly_model.predict(X_test_poly)

mse_poly = mean_squared_error(y_test, y_pred_poly)
r2_poly = r2_score(y_test, y_pred_poly)

print("\nPolynomial Regression Results")
print("MSE:", mse_poly)
print("R2 Score:", r2_poly)

# Optional Visualization (only if single feature dataset)
if X.shape[1] == 1:
    plt.scatter(X, y, label="Actual Data")
    X_sorted = np.sort(X.values, axis=0)
    y_poly_sorted = poly_model.predict(poly.transform(X_sorted))
    plt.plot(X_sorted, y_poly_sorted, label="Polynomial Fit")
    plt.xlabel("Input Feature")
    plt.ylabel("Target")
    plt.title("Polynomial Regression Curve")
    plt.legend()
    plt.show()
```

Observation Table

Model	**Mean Squared Error (MSE)**	**R^2 Score**	**Remarks**
Multiple Linear Regression	_____	_____	**Suitable for linear relationships**
Polynomial Regression (Degree = 2)	_____	_____	**Suitable for nonlinear relationships**

Sample Output

Multiple Linear Regression Results

- MSE: 12.45
- R^2 Score: 0.82

Polynomial Regression Results

- MSE: 8.67
- R^2 Score: 0.89

Result

Thus, predictive models using Multiple Linear Regression and Polynomial Regression were successfully developed on the sample dataset. The performance of both models was evaluated using MSE and R^2 score. It was observed that:

- Multiple Linear Regression is more suitable when the relationship between input features and target is linear.
- Polynomial Regression performs better when the relationship is nonlinear.

Hence, the best model depends on the underlying pattern of the dataset.

Precautions

- Ensure the dataset is clean and free from missing values.
- Select the correct target variable before model training.
- Do not choose an unnecessarily high polynomial degree, as it may lead to overfitting.
- Always split data into training and testing sets before evaluation.
- Use proper feature scaling or normalization if required.

Applications

- House price prediction
- Sales forecasting
- Stock trend approximation
- Demand prediction in business analytics
- Weather and environmental data modeling
- Industrial process optimization

---

13. Viva / Assessment Questions

1. What is regression in machine learning?
2. What is the difference between simple linear regression and multiple linear regression?
3. What is polynomial regression?
4. Why is polynomial regression considered linear in parameters?
5. What is overfitting in polynomial regression?

6. What is the difference between MSE and RMSE?
7. What is R^2 score and why is it important?
8. How do you decide the degree of the polynomial?
9. Which model is better for nonlinear data and why?
10. Can polynomial regression be used with multiple input features?

Conclusion

This practical demonstrates how predictive models can be developed using both linear and nonlinear regression techniques. By implementing Multiple Linear Regression and Polynomial Regression, students can understand how different regression models behave with different types of data. The practical also highlights the importance of selecting the correct model based on the nature of the relationship between variables.

Practical No. 6

Performance Improvement of Regression Models Using Ridge and Lasso Regularization on a Sample Dataset

---

Aim

To analyze and improve the performance of regression models using **Ridge (L2)** and **Lasso (L1)** regularization techniques on a sample dataset.

Objective

- To understand the concept of **regularization** in regression.
- To implement **Multiple Linear Regression**, **Lasso Regression**, and **Ridge Regression**.
- To compare the models using **validation Mean Squared Error (MSE)**.
- To observe the effect of regularization on **model coefficients**.

Theory

Regularization is a technique used to prevent **overfitting** in regression models by adding a penalty term to the loss function.

(a) Linear Regression

A standard linear regression model minimizes the residual sum of squares:

$$\text{Loss} = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

(b) Ridge Regression (L2 Regularization)

Ridge regression adds a penalty proportional to the **square of coefficients**:

$$\text{Loss} = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^{p} w_j^2$$

- Shrinks coefficients toward zero
- Reduces model complexity
- Helps with multicollinearity

(c) Lasso Regression (L1 Regularization)

Lasso regression adds a penalty proportional to the **absolute value of coefficients**:

$$\text{Loss} = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^{p} |w_j|$$

- Can shrink some coefficients exactly to **zero**
- Performs **feature selection**
- Useful when some predictors are irrelevant

In the referenced Harvard exercise, the notebook uses **Boston_housing.csv**, selects 11 predictor columns, normalizes them, splits data into **70% train / 30% validation** with **random_state = 31**, and compares **Linear Regression**, **Lasso(alpha = 0.008)**, and **Ridge(alpha = 0.008)** by validation MSE and coefficient plots.

Requirements

- **Software:** Python 3.x / Jupyter Notebook / Google Colab / VS Code
- **Libraries:** NumPy, Pandas, Matplotlib, Scikit-learn

- **Dataset:** Boston_housing.csv

Predictor Variables (as per reference notebook)

- crim
- indus
- nox
- rm

- age
- dis
- rad
- tax
- ptratio
- black
- lstat

Response Variable

- medv

Algorithm

1. Import the required libraries.
2. Load the Boston_housing.csv dataset.
3. Select predictor variables and response variable.
4. Normalize the predictor variables.
5. Split the dataset into **training** and **validation** sets.

6. Train a **Multiple Linear Regression** model.
7. Predict on the validation set.
8. Compute the **validation MSE**.
9. Store the regression coefficients.
10. Train a **Lasso Regression** model with **alpha = 0.008**.
11. Predict and compute validation MSE for Lasso.
12. Store Lasso coefficients.
13. Train a **Ridge Regression** model with **alpha = 0.008**.
14. Predict and compute validation MSE for Ridge.
15. Store Ridge coefficients.
16. Compare all three models using:

- Validation MSE
- Coefficient comparison graph

17. Interpret the results.

Program (Python Code)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn import preprocessing
from sklearn.linear_model import LinearRegression, Lasso, Ridge
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
```

```
# Load dataset
df = pd.read_csv("Boston_housing.csv")

# Select predictors and target
X = df[['crim', 'indus', 'nox', 'rm', 'age', 'dis', 'rad', 'tax', 'ptratio', 'black', 'lstat']]
X_norm = preprocessing.normalize(X)
y = df['medv']

# Split into train and validation sets
X_train, X_val, y_train, y_val = train_test_split(

    X_norm, y, test_size=0.3, random_state=31
)

# 1. Multiple Linear Regression
lreg = LinearRegression()
lreg.fit(X_train, y_train)
y_pred_lr = lreg.predict(X_val)
mse_lr = mean_squared_error(y_val, y_pred_lr)

# 2. Lasso Regression
lasso_reg = Lasso(alpha=0.008)
lasso_reg.fit(X_train, y_train)
```

```
y_pred_lasso = lasso_reg.predict(X_val)
mse_lasso = mean_squared_error(y_val, y_pred_lasso)

# 3. Ridge Regression
ridge_reg = Ridge(alpha=0.008)
ridge_reg.fit(X_train, y_train)
y_pred_ridge = ridge_reg.predict(X_val)
mse_ridge = mean_squared_error(y_val, y_pred_ridge)

# Print MSE values
print("Linear Regression Validation MSE:", mse_lr)
print("Lasso Regression Validation MSE:", mse_lasso)
print("Ridge Regression Validation MSE:", mse_ridge)

# Coefficient comparison
coef_df = pd.DataFrame({
    'Feature': X.columns,
    'Linear': lreg.coef_,
    'Lasso': lasso_reg.coef_,
    'Ridge': ridge_reg.coef_
})

print("\nCoefficient Comparison:\n")
print(coef_df)
```

```
# Plot coefficients

plt.figure(figsize=(10, 6))

plt.barh(coef_df['Feature'], coef_df['Linear'], height=0.8, label='Linear Regression',
alpha=0.6)

plt.barh(coef_df['Feature'], coef_df['Lasso'], height=0.5, label='Lasso', alpha=0.6)

plt.barh(coef_df['Feature'], coef_df['Ridge'], height=0.2, label='Ridge', alpha=0.6)

plt.xlabel("Coefficient Value")

plt.ylabel("Features")

plt.title("Comparison of Regression Coefficients")

plt.legend()

plt.grid(True, linewidth=0.3)

plt.show()
```

Observation Table

Model	Alpha	Validation MSE	Coefficient Behavior	Remarks
Linear Regression	0	____	Large / unregularized	Baseline
Lasso Regression	0.008	____	Some coefficients shrink / may become 0	Feature selection possible
Ridge Regression	0.008	____	All coefficients shrink smoothly	Stable model

Result

The regression models were successfully implemented and compared.
Ridge and **Lasso** regularization helped control coefficient magnitude and reduce overfitting.
Lasso may eliminate less important features by making some coefficients zero, while **Ridge** reduces all coefficients smoothly.

Precautions

- Normalize or standardize features before applying regularization.
- Use the same train-validation split for all models.
- Choose alpha carefully; a very high alpha may cause **underfitting**.
- Interpret Lasso coefficients carefully because zero values imply feature removal.

Applications

- House price prediction
- Financial forecasting
- Medical score prediction
- Feature selection in high-dimensional data
- Regression under multicollinearity

Viva Questions

1. What is regularization?
2. Why is regularization used in regression?
3. What is the difference between **L1** and **L2** regularization?
4. Why can Lasso perform feature selection?

5. What is the role of **alpha (λ)**?
6. Why should features be scaled before Ridge/Lasso?
7. What is overfitting?
8. What is validation MSE?

Conclusion

This practical demonstrates that **Ridge** and **Lasso** regularization improve regression robustness by reducing overfitting and controlling coefficient magnitude. Lasso additionally helps in feature selection, while Ridge improves stability in the presence of correlated predictors.

REFERENCE LINK:

https://harvard-iacs.github.io/2021-CS109A/lectures/lecture06/notebook-1/

Practical No. 7

Title: Evaluation of Machine Learning Models Using Cross-Validation and Standard Performance Metrics on a Sample Dataset

---

Aim

To evaluate machine learning models using **simple validation**, **k-fold cross-validation**, and **standard performance metrics** on a sample dataset.

Objective

- To understand **train-validation splitting**.
- To implement **k-fold cross-validation**.
- To compare **simple validation** with **cross-validation**.
- To use **Mean Squared Error (MSE)** as a standard performance metric.
- To identify the best regularization parameter (**alpha**) for Ridge regression.

Theory

Model evaluation is necessary to estimate how well a model will perform on unseen data.

(a) Simple Validation

The dataset is split into:

- Training set
- Validation set

The model is trained on the training set and evaluated on the validation set.

(b) K-Fold Cross-Validation

In **k-fold cross-validation**:

1. The data is divided into **k equal parts**.
2. The model is trained on **k−1 folds**.
3. It is validated on the remaining fold.
4. This process is repeated **k times**.
5. The average validation score is taken.

(c) Mean Squared Error (MSE)

For regression problems:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

A lower MSE indicates better predictive performance.

In the referenced Harvard exercise:

- Dataset: **polynomial50.csv**
- Polynomial degree: **30**
- Alpha values: **[1e-7, 1e-5, 1e-3, 0.01, 0.1, 1]**
- Simple validation uses **80% train**
- Cross-validation uses **5 folds**
- Multiple random states are tested to compare best alpha and best MSE.

Requirements

- **Software:** Python 3.x / Jupyter Notebook / Google Colab / VS Code

- **Libraries:** NumPy, Pandas, Matplotlib, Scikit-learn
- **Dataset:** polynomial50.csv

Columns Used

- x → predictor
- y → response
- f → true function (for visualization)

Algorithm

1. Import required libraries.
2. Define:
    - List of random states
    - List of alpha values
    - Polynomial degree = 30
3. Load the polynomial50.csv dataset.
4. Select predictor x, response y, and true function f.
5. Create polynomial features of degree 30.
6. Perform **simple validation**:
    - Split data into **80% training** and **20% validation**
    - Train Ridge regression for each alpha
    - Compute training and validation MSE
7. Determine the best alpha based on minimum validation MSE.

8. Perform **5-fold cross-validation**:
    - For each alpha, compute average training MSE
    - Compute average cross-validation MSE

9. Select the best alpha based on minimum CV MSE.
10. Compare:

- Training error
- Validation error
- Cross-validation error

11. Plot results and interpret them.

Program (Python Code)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split, cross_validate
from sklearn.preprocessing import PolynomialFeatures

# Parameters
ran_state = [0, 10, 21, 42, 66, 109, 310, 1969]
alphas = [1e-7, 1e-5, 1e-3, 0.01, 0.1, 1]
degree = 30
```

```
# Load data
df = pd.read_csv("polynomial50.csv")
x = df[['x']].values

y = df['y'].values
f = df['f'].values
# Function: Simple Validation
def reg_with_validation(rs):
    x_train, x_val, y_train, y_val = train_test_split(
        x, y, train_size=0.8, random_state=rs
    )

    training_error, validation_error = [], []

    poly = PolynomialFeatures(degree)
    x_poly_train = poly.fit_transform(x_train)
    x_poly_val = poly.transform(x_val)

    for alpha in alphas:
        ridge_reg = Ridge(alpha=alpha, fit_intercept=False)
        ridge_reg.fit(x_poly_train, y_train)

        y_train_pred = ridge_reg.predict(x_poly_train)
        y_val_pred = ridge_reg.predict(x_poly_val)
```

```
        mse_train = mean_squared_error(y_train, y_train_pred)
        mse_val = mean_squared_error(y_val, y_val_pred)
training_error.append(mse_train)
        validation_error.append(mse_val)

    return training_error, validation_error

# Function: Cross Validation
def reg_with_cross_validation(rs):
    df_new = df.sample(frac=1, random_state=rs)
    x_new = df_new[['x']].values
    y_new = df_new['y'].values

    poly = PolynomialFeatures(degree)
    x_poly = poly.fit_transform(x_new)

    training_error, cv_error = [], []

    for alpha in alphas:
        ridge_reg = Ridge(alpha=alpha, fit_intercept=False)

        scores = cross_validate(
            ridge_reg,
            x_poly,
```

```
y_new,

cv=5,
        scoring='neg_mean_squared_error',
        return_train_score=True
    )
    mse_train = -np.mean(scores['train_score'])



 mse_cv = -np.mean(scores['test_score'])
    training_error.append(mse_train)
    cv_error.append(mse_cv)
  return training_error, cv_error
# Example run for one random state
rs = 42
train_err_val, val_err = reg_with_validation(rs)
train_err_cv, cv_err = reg_with_cross_validation(rs)

print("Simple Validation Results:")
for a, tr, va in zip(alphas, train_err_val, val_err):
  print(f"Alpha={a}, Train MSE={tr:.6f}, Validation MSE={va:.6f}")
print("\nCross-Validation Results:")
for a, tr, cv in zip(alphas, train_err_cv, cv_err):
  print(f"Alpha={a}, Train MSE={tr:.6f}, CV MSE={cv:.6f}")
```

```
# Plot simple validation
plt.figure(figsize=(8,5))
plt.plot(alphas, train_err_val, 'o--', label='Training Error')
plt.plot(alphas, val_err, 'o-', label='Validation Error')
plt.xscale('log')
plt.xlabel("Alpha")
plt.ylabel("MSE")


plt.title("Simple Validation: Ridge Regression")
plt.legend()
plt.grid(True)
plt.show()

# Plot cross-validation
plt.figure(figsize=(8,5))
plt.plot(alphas, train_err_cv, 'o--', label='Training Error')
plt.plot(alphas, cv_err, 'o-', label='Cross-Validation Error')
plt.xscale('log')
plt.xlabel("Alpha")
plt.ylabel("MSE")
plt.title("5-Fold Cross-Validation: Ridge Regression")
plt.legend()
```

```
plt.grid(True)

plt.show()
```

7. Observation Table

Alpha	Training MSE	Validation MSE	Cross-Validation MSE	Best Alpha?
1e-7	_____	_____	_____	_____
1e-5	_____	_____	_____	_____
1e-3	_____	_____	_____	_____
0.01	_____	_____	_____	_____
0.1	_____	_____	_____	_____
1	_____	_____	_____	_____

Sample Output

- Training MSE for each alpha
- Validation MSE for each alpha
- Cross-validation MSE for each alpha
- Best alpha based on minimum validation/CV error
- Error vs alpha plots on a logarithmic scale

Result

The model was successfully evaluated using both **simple validation** and **5-fold cross-validation**. Cross-validation produced a more stable and reliable estimate of model performance compared to a single train-validation split.

Precautions

- Do not use the test set for tuning hyperparameters.
- Use the same polynomial degree while comparing alphas.
- Use proper scoring metrics for the problem type.
- Use random_state for reproducibility.
- Cross-validation should be performed on training data or during model selection.

Applications

- Hyperparameter tuning
- Model selection
- Reliable model evaluation
- Comparing algorithms fairly
- Regression and classification workflows

Viva Questions

1. What is model evaluation?
2. What is the difference between training error and validation error?
3. What is cross-validation?
4. Why is **k-fold cross-validation** preferred over a single split?
5. What is MSE?
6. What is hyperparameter tuning?

7. Why can a single validation split be misleading?
8. What is the purpose of choosing the best alpha?

Conclusion

This practical demonstrates that **cross-validation** is a more reliable model evaluation technique than simple validation. It reduces the dependency on a single random split and helps select better hyperparameters for improved generalization.

REFERENCE LINK:

https://harvard-iacs.github.io/2021-CS109A/lectures/lecture06/notebook-5/

Practical No. 8

Title: Analysis of Overfitting and Underfitting Through the Bias–Variance Trade-Off Using a Sample Dataset

---

Aim

To analyze **overfitting** and **underfitting** through the **bias–variance trade-off** using polynomial regression on a sample dataset.

Objective

- To understand the concepts of **bias** and **variance**.
- To compare **underfitting** and **overfitting**.
- To fit polynomial models of different degrees.
- To observe how model complexity affects generalization.

Theory

The **bias–variance trade-off** explains the relationship between model complexity and prediction behavior.

(a) High Bias (Underfitting)

A model with very low complexity may fail to capture the underlying pattern in data.

- Too simple
- Poor fit on training and test data
- Example: Low-degree polynomial (e.g., degree 1)

(b) High Variance (Overfitting)

A model with very high complexity may fit noise in the training data.

- Too complex

- Excellent fit on training data
- Poor generalization on new data
- Example: Very high-degree polynomial (e.g., degree 100)

(c) Balanced Model

A good model finds the right balance between:

- Low bias
- Low variance
- Good generalization

In the referenced Harvard exercise:

- Dataset: **noisypopulation.csv**
- Predictor: x
- Response: y
- True function: f
- Repeated sampling is performed
- **200 samples** are generated
- **30 points** are selected in each sample
- Models are compared for **degree 1** and **degree 100** to visualize the bias–variance trade-off.

Requirements

- **Software:** Python 3.x / Jupyter Notebook / Google Colab / VS Code
- **Libraries:** NumPy, Pandas, Matplotlib, Scikit-learn
- **Dataset:** noisypopulation.csv

Columns Used

- x → predictor
- y → response
- f → true function

Algorithm

1. Import the required libraries.
2. Load the noisypopulation.csv dataset.
3. Select predictor x, response y, and true function f.
4. Define a function to generate polynomial features for a chosen degree.
5. Define a function to fit linear regression on transformed polynomial features.
6. Repeatedly sample random subsets from the dataset.
7. For each sample:
    - Fit a **degree 1** polynomial model
    - Predict over the full input range
8. Repeat the process for a **degree 100** polynomial model.
9. Plot all sampled predictions for both models.
10. Compare:

- Stability
- Flexibility
- Underfitting vs overfitting

11. Interpret the bias–variance trade-off.

Program (Python Code)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures

# Load dataset
df = pd.read_csv("noisypopulation.csv")

# Predictor, response, and true function
x = df['x'].values
y = df['y'].values
f = df['f'].values

# Function to generate polynomial features
def polyshape(degree, xvals):
    return PolynomialFeatures(degree).fit_transform(xvals.reshape(-1, 1))

# Function to fit model and predict
def make_predict_with_model(x_train, y_train, x_pred):
    lreg = LinearRegression(fit_intercept=False)
    lreg.fit(x_train, y_train)
    y_pred = lreg.predict(x_pred)
    return lreg, y_pred

# Sampling function
def gen(degree, num_sample, size, x, y):
    predicted_values, linear_models = [], []

    for _ in range(num_sample):
        indexes = np.sort(np.random.choice(x.shape[0], size=size, replace=False))
        lreg, y_pred = make_predict_with_model(
```

```
 polyshape(degree, x[indexes]),

 y[indexes],
        polyshape(degree, x)
      )
      predicted_values.append(y_pred)
      linear_models.append(lreg)

   return predicted_values, linear_models



# Generate predictions for degree 1 and degree 100
predicted_1, model_1 = gen(1, 200, 30, x, y)
predicted_100, model_100 = gen(100, 200, 30, x, y)

# Plotting
indexes = np.sort(np.random.choice(x.shape[0], size=30, replace=False))

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Degree 1
axes[0].plot(x, f, label='True Function', color='blue', linewidth=3)
axes[0].plot(x, y, '.', label='Population y', alpha=0.5)
axes[0].plot(x[indexes], y[indexes], 's', color='black', label='Sampled Data')

for pred in predicted_1[:-1]:
   axes[0].plot(x, pred, alpha=0.03, color='orange')
axes[0].plot(x, predicted_1[-1], alpha=0.5, color='orange', label='Degree 1 Models')

axes[0].set_title("Degree 1 Polynomial (High Bias, Low Variance)")
axes[0].legend()

# Degree 100
axes[1].plot(x, f, label='True Function', color='blue', linewidth=3)
axes[1].plot(x, y, '.', label='Population y', alpha=0.5)
axes[1].plot(x[indexes], y[indexes], 's', color='black', label='Sampled Data')
```

```
for pred in predicted_100[:-1]:

axes[1].plot(x, pred, alpha=0.03, color='red')
axes[1].plot(x, predicted_100[-1], alpha=0.5, color='red', label='Degree 100 Models')

axes[1].set_title("Degree 100 Polynomial (Low Bias, High Variance)")
axes[1].legend()

plt.tight_layout()
plt.show()
```

Observation Table

Model Degree	No. of Samples	Points per Sample	Observed Behavior	Bias / Variance	Remarks
1	200	30	Smooth / stable / underfits	High Bias, Low Variance	Simple model
100	200	30	Highly fluctuating / overfits	Low Bias, High Variance	Complex model

---

Sample Output

- Plot of repeated predictions for **degree 1**
- Plot of repeated predictions for **degree 100**
- Visual demonstration of:
  - Underfitting
  - Overfitting
  - Bias–variance trade-off

Result

The practical successfully demonstrated the **bias–variance trade-off**.

- **Degree 1** polynomial showed **high bias** and **low variance** (underfitting).
- **Degree 100** polynomial showed **low bias** and **high variance** (overfitting).

Precautions

- Use enough repeated samples to clearly visualize variance.
- Keep the same dataset while changing only model complexity.
- Do not judge a model only by training fit.
- Use plots to support interpretation.
- Extremely high polynomial degrees may become numerically unstable.

Applications

- Model selection
- Choosing polynomial degree
- Understanding generalization error
- Hyperparameter tuning
- Designing robust ML pipelines

Viva Questions

1. What is bias in machine learning?
2. What is variance in machine learning?
3. What is underfitting?
4. What is overfitting?

5. Why does a high-degree polynomial overfit?
6. Why does a low-degree polynomial underfit?
7. What is the bias–variance trade-off?
8. How can regularization reduce variance?

Conclusion

This practical clearly shows that increasing model complexity reduces bias but can increase variance. Therefore, the best model is not necessarily the most complex one; instead, it should achieve a balance between **fit quality** and **generalization ability**.

REFERENCE LINK:

https://harvard-iacs.github.io/2021-CS109A/lectures/lecture06/notebook-2/

Practical No. 9

Title: Implementation of a Single-Layer Perceptron for Binary Classification Using a Sample Dataset

Aim

To implement a Single-Layer Perceptron (SLP) for solving a binary classification problem using a sample dataset.

Objective

- To understand the working of the Perceptron Learning Algorithm
- To perform binary classification
- To train a model using labeled data
- To evaluate model performance using accuracy and classification metrics

Theory

A Single-Layer Perceptron is the simplest artificial neural network model used for binary classification. It contains an input layer directly connected to an output neuron. The model computes a weighted sum of inputs and applies a step activation function. It works effectively only for linearly separable data. The weights and bias are updated using the perceptron learning rule until the error is minimized.

Requirements

- Python 3.x
- Jupyter Notebook / Google Colab / VS Code
- Libraries: NumPy, Scikit-learn
- Sample binary dataset: Breast Cancer / Iris (2 classes) / AND / OR dataset

Algorithm

1. Import required libraries
2. Load the sample dataset
3. Split the dataset into training and testing sets
4. Normalize the input features
5. Initialize weights and bias to zero
6. Train the perceptron for a fixed number of epochs
7. Predict outputs for the test set
8. Evaluate the model using accuracy and confusion matrix
9. Display the results

Flowchart (Textual Representation)

1. Start
2. Load dataset
3. Preprocess / normalize data
4. Split into training and testing data
5. Initialize model parameters
6. Train the model
7. Generate predictions
8. Evaluate performance
9. Display output
10. Stop

Program

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

data = load_breast_cancer()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

class Perceptron:
    def __init__(self, learning_rate=0.01, epochs=100):
        self.lr = learning_rate
        self.epochs = epochs
        self.weights = None
        self.bias = None
```

```
def activation(self, x):
    return np.where(x >= 0, 1, 0)

  def fit(self, X, y):
    n_samples, n_features = X.shape
    self.weights = np.zeros(n_features)
    self.bias = 0

    for _ in range(self.epochs):
      for i in range(n_samples):
        linear_output = np.dot(X[i], self.weights) + self.bias
        y_pred = self.activation(linear_output)



update = self.lr * (y[i] - y_pred)



self.weights += update * X[i]
        self.bias += update

  def predict(self, X):
    linear_output = np.dot(X, self.weights) + self.bias
    return self.activation(linear_output)

model = Perceptron(learning_rate=0.01, epochs=100)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

Observation Table

Epoch / Iteration	Training Accuracy	Testing Accuracy	Remarks

Sample Output

Accuracy obtained is generally between 94% and 98% depending on the dataset split. The confusion matrix and classification report are displayed.

Result

The Single-Layer Perceptron was successfully implemented and used for binary classification. The model achieved satisfactory performance on the sample dataset.

Precautions

- Ensure the dataset contains only two classes for binary classification
- Normalize input features before training
- Use a suitable learning rate to avoid unstable updates
- Do not use SLP for non-linearly separable problems like XOR

Applications

- Spam email detection
- Medical diagnosis (benign vs malignant)
- Credit approval systems
- Basic pattern recognition

Viva Questions

- What is a perceptron?
- Why is a single-layer perceptron used only for linear classification?
- What is the role of the activation function?
- What is the perceptron learning rule?
- Why can a perceptron not solve XOR?
- What is the difference between weights and bias?
- What is binary classification?

Conclusion

This experiment demonstrates the implementation of a Single-Layer Perceptron for linearly separable binary classification tasks and builds the foundation for understanding advanced neural network models.

Practical No. 10

Title : Design and Implementation of a Multilayer Perceptron Using the Backpropagation Algorithm on a Sample Dataset

Aim

To design and implement a Multilayer Perceptron (MLP) using the backpropagation algorithm for classification on a sample dataset.

Objective

- To understand the architecture of a Multilayer Perceptron
- To learn the concept of forward propagation and backpropagation
- To classify data using hidden layers
- To compare the performance of MLP with simple perceptron

Theory

A Multilayer Perceptron (MLP) is a feedforward artificial neural network consisting of an input layer, one or more hidden layers, and an output layer. Unlike a single-layer perceptron, an MLP can solve non-linear classification problems. It uses non-linear activation functions and learns by backpropagation, where the error is propagated backward and weights are updated using gradient descent or its variants.

Requirements

- Python 3.x
- Jupyter Notebook / Google Colab / VS Code
- Libraries: NumPy, Scikit-learn
- Sample dataset: Breast Cancer / Iris / MNIST subset

Algorithm

1. Import required libraries
    - Load the sample dataset
    - Split the dataset into training and testing sets
    - Normalize the input features
    - Create an MLP model with hidden layers
    - Train the model using backpropagation
    - Predict outputs for the test set
    - Evaluate the model using accuracy and confusion matrix
    - Display the results

Flowchart (Textual Representation)

1. Start
2. Load dataset
3. Preprocess / normalize data
4. Split into training and testing data
5. Initialize model parameters
   Train the model
6. Generate predictions
7. Evaluate performance
8. Display output
9. Stop

Program

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

data = load_breast_cancer()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

mlp = MLPClassifier(
    hidden_layer_sizes=(16, 8),
    activation='relu',
    solver='adam',
```

```
 learning_rate_init=0.001,

max_iter=500,
   random_state=42
)

mlp.fit(X_train, y_train)
y_pred = mlp.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

Observation Table

Epoch / Iteration	Training Accuracy	Testing Accuracy	Remarks

Sample Output

Accuracy obtained is generally between 95% and 99% depending on network architecture and dataset. The confusion matrix and classification report are displayed.

Result

The Multilayer Perceptron was successfully implemented using backpropagation and produced better performance than a simple perceptron for complex classification tasks.

Precautions

- Always normalize features before training an MLP
- Choose hidden layers carefully to avoid overfitting
- Use sufficient iterations for convergence
- Monitor learning rate and training loss

Applications

- Image recognition
- Handwritten digit classification
- Speech recognition
- Medical diagnosis and decision support

Viva Questions

10. What is a Multilayer Perceptron?
11. What is the role of hidden layers?
12. What is backpropagation?
13. Why is MLP better than a single-layer perceptron?
14. What are activation functions?
15. What is gradient descent?
16. What is the difference between forward propagation and backpropagation?
17. Why is feature scaling important in MLP?

Conclusion

This experiment shows that a Multilayer Perceptron trained with backpropagation can learn complex and non-linearly separable patterns more effectively than a single-layer perceptron.